

EVENTFORCE MANAGEMENT SYSTEM

Salesforce Implementation Project Documentation

Program: Salesforce Developer

Team Leader: Kiruthika.K

Team Members: Madhumitha.M, Janani.S, Kalki.K

Registered Mail ID: kiruthikakannan061@gmail.com

Team ID: SWTID-2026-4043

College: Arasu Engineering College

Platform: Salesforce Developer Edition Org (Lightning Experience)

Submission Date: 3rd October 2026

Contents

1. Project Overview
 2. Objectives
 3. Ideation Phase
 4. Requirement Analysis
 5. Project Design
 6. Project Planning
 7. Phase 1: Functional Scope and Execution Roadmap
 8. Phase 2: Backend Development and Configurations
 9. Phase 3: UI/UX Development and Customization
 10. Phase 4: Data Migration, Testing and Security
 11. Phase 5: Deployment, Documentation and Maintenance
 12. Challenges Faced and Fixes
 13. Conclusion
- Appendix A: Apex Source Code

1. Project Overview

EventForce Management System is a CRM application built on the Salesforce platform for an event planning company. Without such a system, details about clients, bookings, venues, vendors and feedback are usually scattered across spreadsheets, phone calls and messages. This project brings all of that into one Salesforce org so that every team member works from the same up-to-date record.

The application is made up of six custom objects (Event, Client, Vendor, Venue, Feedback and Event Vendor), relationships that link them, a validation rule that protects data quality, a formula field that calculates the event budget, an approval process for event cancellations, a record-triggered Flow that sends reminders, and Apex code that keeps venue availability correct, blocks double bookings and completes past events every night. A Lightning App, a report and a dashboard give a clear view of upcoming events. Access is controlled with roles, profiles, a permission set, organization-wide defaults and a sharing rule.

The whole solution was configured and tested in a Salesforce Developer Edition org, following the order a real consultant would follow: gather requirements, model the data, automate the process, design the interface, secure access, test and document.

2. Objectives

- Keep client, event, venue, vendor and feedback information in one well-structured place.
- Reduce manual work in booking events and reserving venues.
- Stop two events from being booked at the same venue on the same date.
- Send clients an automatic reminder before their event.
- Route event cancellation requests through a controlled approval process.
- Give each type of user access only to what they need.
- Give management clear visibility of upcoming events through a report and a dashboard.

3. Ideation Phase

3.1 Problem Statement

Event planning companies handle many clients, venues and vendors at the same time. When this information lives in spreadsheets and chats, bookings are missed, venues get double-booked and clients are not kept informed. The table below describes the problem from the point of view of two users.

Item	Event Coordinator	Client
I am	An event coordinator at an event planning company	A customer who books an event
I am trying to	Book events, reserve venues and track every booking in one place	Book an event and know that everything is confirmed
But	Records are scattered and I can book the same venue twice by mistake	I do not get reminders or updates about my event
Because	There is no central system with rules and automation	Communication is manual and depends on the coordinator
Which makes me feel	Stressed and worried about mistakes	Uncertain and unimportant

3.2 Empathy Map

The empathy map describes the main user, the Event Coordinator, so that the solution is built around what this person actually experiences.

Quadrant	Event Coordinator
Says	"I need to know which venues are free." "Did we send the reminder to the client?" "Can I cancel this booking quickly?"
Thinks	Double booking will damage the company's reputation. Too much time is lost in manual follow-ups.
Does	Maintains spreadsheets, calls vendors and venues, sends reminders by hand, asks managers to approve cancellations.
Feels	Pressure to avoid mistakes, frustrated with repeated work, relieved when everything is confirmed.
Pains	Double bookings, missed reminders, no single view of upcoming events, slow approvals.
Gains	Automatic checks, automatic reminders, a clear dashboard and a controlled approval process.

3.3 Brainstorming and Idea Prioritization

The team listed possible features and ranked them by business impact and effort. Features with high impact and low to medium effort were selected for the project.

Idea	Impact	Effort	Decision
Central objects for events, clients, venues, vendors and feedback	High	Low	Selected
Prevent double booking of a venue on the same date	High	Medium	Selected
Automatic email reminder before the event	High	Low	Selected
Approval process for event cancellation	High	Medium	Selected

Idea	Impact	Effort	Decision
Automatic event budget from the event type	Medium	Low	Selected
Nightly job that marks past events as completed	Medium	Medium	Selected
Report and dashboard of upcoming events	High	Low	Selected
Role-based access for coordinators, vendor managers and clients	High	Medium	Selected
Online payment integration	Medium	High	Future scope
Mobile app for clients	Medium	High	Future scope

4. Requirement Analysis

4.1 Customer Journey

Stage	What the user does	Pain point	How EventForce helps
Enquiry	Client contacts the company about an event	Details are noted on paper or chat	Client and event records are created in Salesforce
Planning	Coordinator selects a venue and date	Venue may already be booked	Double-booking trigger blocks the save and shows an error
Confirmation	Event is confirmed	Venue status is updated manually	Venue becomes Reserved automatically
Before the event	Client must be reminded	Reminders are forgotten	Flow sends an email three days before the event
Cancellation	Client or coordinator asks to cancel	No control over who approves	Approval process with email alerts
After the event	Event is closed and feedback is collected	Past events stay open	Nightly batch marks them Completed; Feedback object stores ratings

4.2 Solution Requirements

Functional requirements

ID	Requirement
FR-1	Store clients, events, venues, vendors and feedback as separate records with relationships.
FR-2	Validate the client email address before saving.
FR-3	Calculate the event budget automatically from the event type.
FR-4	Prevent two events from being booked at the same venue on the same date.
FR-5	Update the venue availability when an event is confirmed or canceled.
FR-6	Send an automatic reminder email to the client before the event.
FR-7	Route cancellation requests through an approval process.

ID	Requirement
FR-8	Mark past events as Completed every night.
FR-9	Provide a report and a dashboard of upcoming events.
FR-10	Restrict access by role, profile, permission set and sharing rules.

Non-functional requirements

- **Usability:** a simple Lightning App with one tab for each object.
- **Security:** private organization-wide default on events with controlled sharing.
- **Reliability:** automation runs on the platform without manual steps.
- **Maintainability:** declarative tools first, Apex only where needed, with clear naming.

4.3 Data Flow

Step	Input	Process	Output / data store
1	Client details	Email validation rule	Client record
2	Event details, client, venue	Double-booking trigger, budget formula	Event record
3	Confirmed event	Venue helper trigger	Venue status set to Reserved
4	Confirmed event date	Scheduled path in the Flow	Reminder email to the client
5	Status Pending Cancellation	Approval process	Status Canceled or Rejected, email alert
6	Past events	Nightly batch Apex	Status Completed
7	Event records	Report and dashboard	Upcoming events by month

4.4 Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition, Lightning Experience
Data model	Custom objects, fields, lookup and master-detail relationships
Declarative automation	Validation rule, formula field, record-triggered Flow, approval process
Programmatic	Apex classes and triggers, Batch Apex and Schedulable Apex
User interface	Lightning App, report, dashboard
Security	Roles, profiles, permission set, organization-wide defaults, sharing rule
Tools	Setup, Developer Console, Flow Builder, GitHub

5. Project Design

5.1 Problem-Solution Fit

Problem	Solution in EventForce
Scattered information	One org with linked objects for clients, events, venues, vendors and feedback
Wrong data entry	Email validation rule and an automatic budget formula
Double booking	Apex trigger that blocks the same venue on the same date
Forgotten reminders	Record-triggered Flow with a scheduled path
Uncontrolled cancellations	Approval process with record lock and email alerts
No visibility	Report and dashboard for upcoming events

5.2 Proposed Solution

Item	Description
Solution	A Salesforce application, EventForce, that manages the complete life cycle of an event: booking, venue reservation, reminders, cancellation approval, completion and feedback.
Novelty	Business rules (double booking, venue status, nightly completion) run automatically, so coordinators do not have to remember them.
Business value	Fewer booking errors, less manual follow-up and clearer visibility for managers.
Scalability	New object types, flows and reports can be added without redesign; the batch job already processes records in chunks of 200.

5.3 Solution Architecture

The solution has three layers. The user layer is the Event Planner Lightning App with tabs, a report and a dashboard. The logic layer holds the validation rule, formula field, approval process, Flow, Apex triggers and the scheduled batch job. The data layer holds the six custom objects. Security (roles, profiles, permission set and sharing) controls what each user sees at every layer.

Layer	Components
User layer	Event Planner app, Events, Clients, Vendors, Venues, Feedbacks, Reports, Dashboards
Logic layer	Email_Valid_Address rule, Event Budget formula, Event_Cancellation_Process, Event_Reminder_Flow, EventTrigger13, PreventDoubleBooking, BatchCompleteEvents, ScheduleCompleteEvents
Data layer	Event__c, Client__c, Vendor__c, Venue__c, Feedback__c, Event_Vendor__c

Layer	Components
Security layer	Roles, profiles, Feedback Manager permission set, organization-wide defaults, Event sharing rule

6. Project Planning

The work was planned in sprints. The table lists each user story with its story points, priority, the team member who worked on it and the dates.

Sprint	User story	Points	Priority	Team member	Start	End
1	As an admin, I create the objects, tabs, fields and relationships	5	High	Kiruthika K	29 Sep 2026	29 Sep 2026
1	As a coordinator, I get email validation and an automatic budget	3	High	Kiruthika K	29 Sep 2026	29 Sep 2026
2	As a manager, I approve or reject cancellations	5	High	Kiruthika K	29 Sep 2026	30 Sep 2026
2	As a client, I receive a reminder before my event	3	High	Kiruthika K	29 Sep 2026	30 Sep 2026
2	As a coordinator, I cannot double-book a venue	5	High	Kiruthika K	29 Sep 2026	30 Sep 2026
2	As a system, I complete past events every night	3	Medium	Kiruthika K	29 Sep 2026	2 Oct 2026
3	As a manager, I see upcoming events in a report and dashboard	5	Medium	Madhumitha M	29 Sep 2026	29 Sep 2026
3	As a user, I work in a Lightning App	2	Medium	Madhumitha M	29 Sep 2026	29 Sep 2026
4	As an admin, I secure access with roles, profiles, permission set and sharing	5	High	Janani S	2 Oct 2026	2 Oct 2026
4	As a tester, I verify every feature in the user interface	3	High	Janani S	2 Oct 2026	2 Oct 2026
5	As a team, we prepare documentation and the demo video	3	High	Kalki K	2 Oct 2026	3 Oct 2026

Total planned story points: 42 across 5 sprints. All sprints were completed by 3 October 2026.

7. Phase 1: Functional Scope and Execution Roadmap

This phase defined what the system must do, who will use it and in what order it would be built.

7.1 Functional Scope

- **Data model:** custom objects Event, Client, Vendor, Venue, Feedback and Event Vendor.
- **Relationships:** lookup and master-detail relationships between the objects.
- **Automation:** a Flow for client reminders, an approval process for cancellations, and Apex for venue logic and nightly processing.
- **Data quality:** a validation rule, a formula field for the event budget and a lookup filter on Feedback.
- **Security:** roles, profiles, a permission set, organization-wide defaults and sharing rules.
- **Analytics:** a report and a dashboard for upcoming events.

7.2 Stakeholder Mapping

Stakeholder	Responsibility in the system
Client	Requests event services, receives reminders and gives feedback.
Event Coordinator	Creates and manages events, communicates with clients and takes part in cancellation approvals.
Vendor Manager	Coordinates vendors such as catering, decor and photography for events.
Venue Manager	Tracks venue availability and reservations.
Event Administrator	Owens the system: security, configuration, approvals and monitoring.

7.3 Execution Roadmap

1. Create the objects, fields and relationships.
2. Add the validation rule and the formula field.
3. Build the approval process for event cancellation.
4. Build the reminder Flow.
5. Write Apex classes and triggers for venue availability, double-booking prevention and nightly event completion.
6. Create the Lightning App, report and dashboard.
7. Create roles, profiles, users, permission set and sharing settings.
8. Test every feature and prepare the documentation and demo.

8. Phase 2: Backend Development and Configurations

This phase built the working core of the application: the data model, the rules that protect it and the automation that runs on it.

8.1 Salesforce Developer Org

A free Developer Edition org was created from developer.salesforce.com, the account was activated through the verification email and a password was set. All configuration in this project was done inside this org.

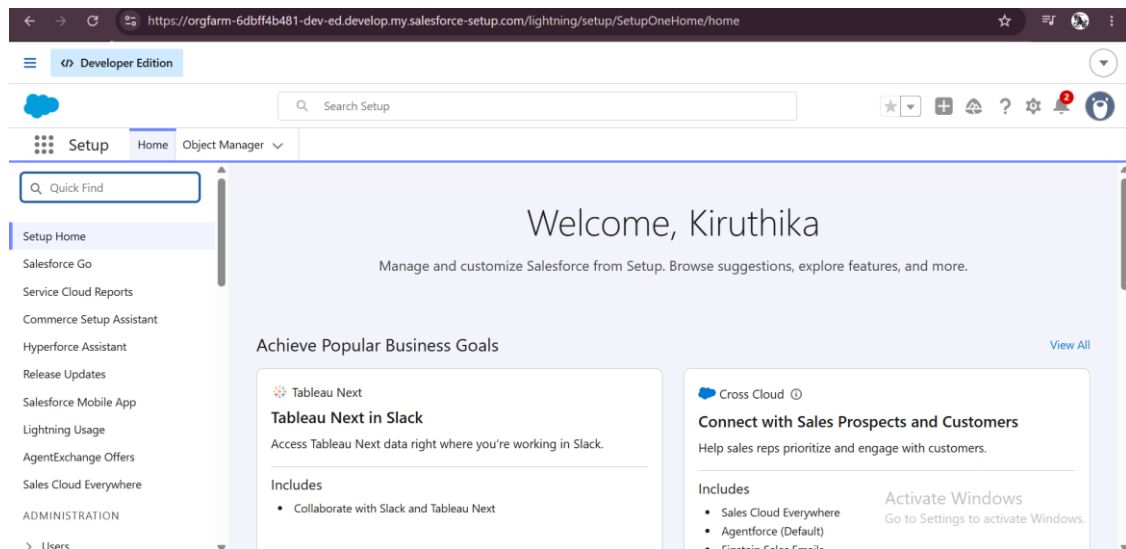


Figure 1: Setup home page of the Developer Edition org

8.2 Data Model

Six custom objects hold the data of the application.

Object	Purpose	Key fields
Event (Event__c)	One record per booked event.	Event Name, Event Date, Event Type, Event Status, Event Budget, Client, Venue
Client (Client__c)	The customer who books the event.	Client Name, Email, Phone, Address, Country, City
Vendor (Vendor__c)	Service providers used in events.	Vendor Name, Email, Phone, Service Type, Status
Venue (Venue__c)	Places where events are held.	Venue Name, Address, Location, Capacity, Availability Status
Feedback (Feedback__c)	Client feedback on an event.	Feedback Name (auto number), Rating, Comments, Event, Client
Event Vendor (Event_Vendor__c)	Records the vendor services linked to an event.	Event Vendor Name, Event (master-detail)

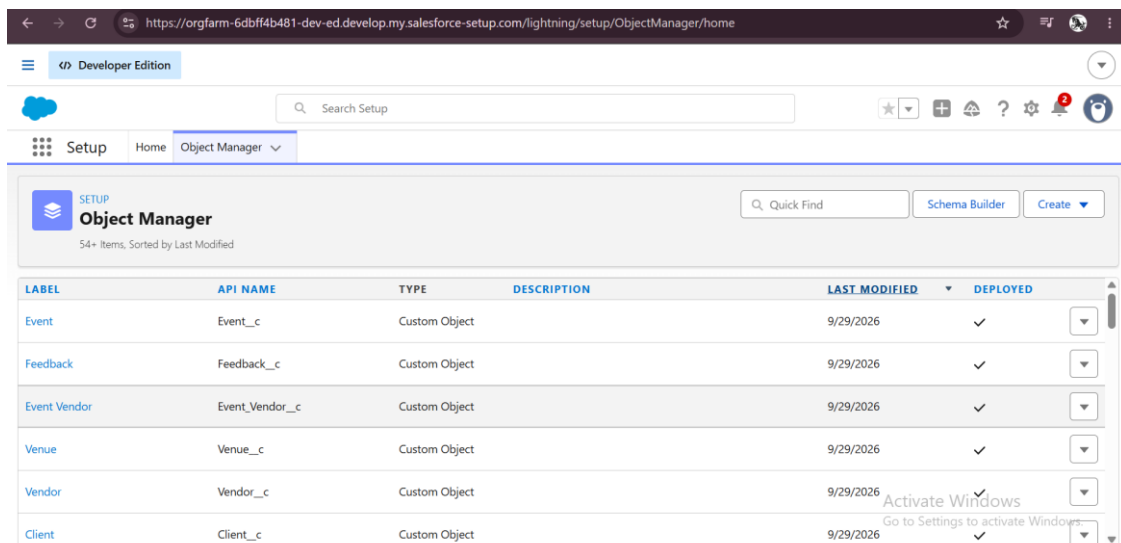
Relationships

From	To	Type
Event	Client	Lookup
Event	Venue	Lookup
Feedback	Event	Lookup
Feedback	Client	Lookup

From	To	Type
Event Vendor	Event	Master-Detail

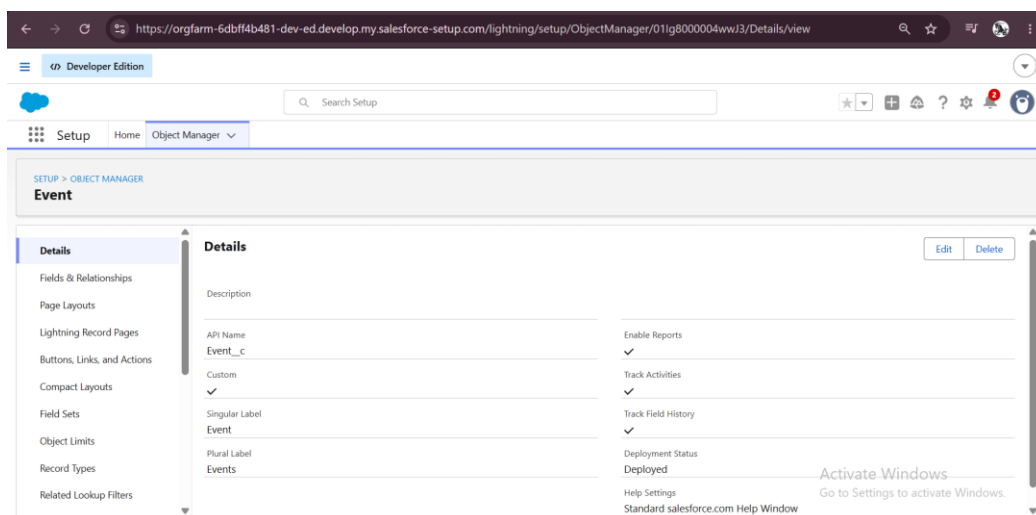
8.3 Creating the Objects

Each object was created from Setup, Object Manager, Create, Custom Object. The label, plural label and record name were entered for every object, and the deployment status was kept as Deployed. For Event, Client, Vendor, Venue and Event Vendor, the options Allow Reports, Track Activities and Track Field History were enabled. Feedback uses an auto-number record name and has Allow Reports enabled.



LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
Event	Event__c	Custom Object		9/29/2026	✓
Feedback	Feedback__c	Custom Object		9/29/2026	✓
Event Vendor	Event_Vendor__c	Custom Object		9/29/2026	✓
Venue	Venue__c	Custom Object		9/29/2026	✓
Vendor	Vendor__c	Custom Object		9/29/2026	✓
Client	Client__c	Custom Object		9/29/2026	✓

Figure 2: Object Manager showing the six custom objects



Details	
Description	
API Name	Event__c
Custom	✓
Singular Label	Event
Plural Label	Events
Enable Reports	✓
Track Activities	✓
Track Field History	✓
Deployment Status	Deployed

Figure 3: Event object details

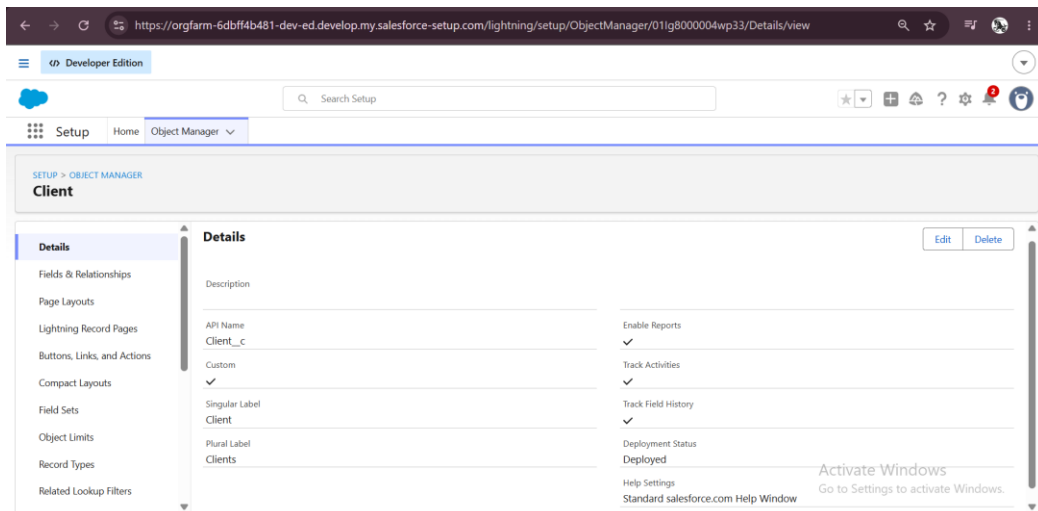


Figure 4: Client object details

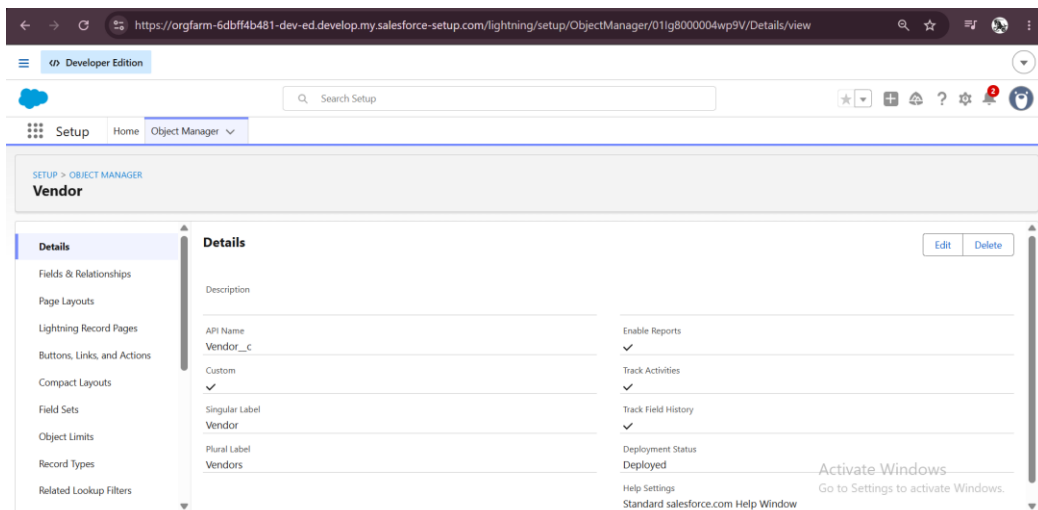


Figure 5: Vendor object details

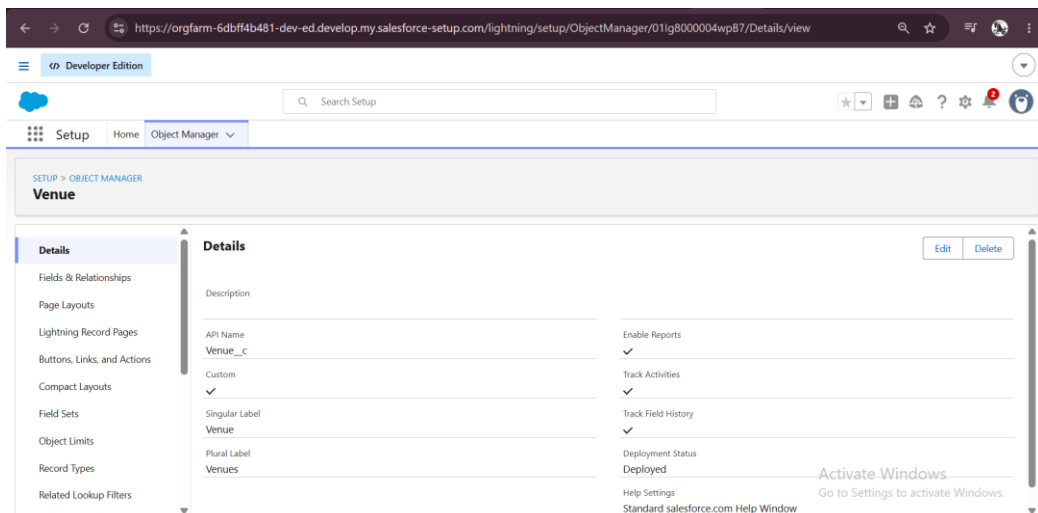


Figure 6: Venue object details

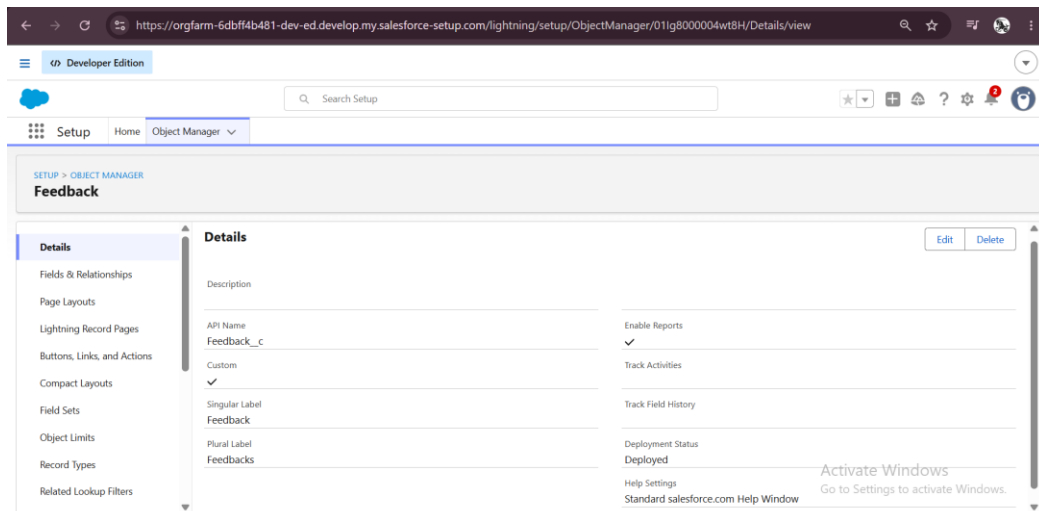


Figure 7: Feedback object details

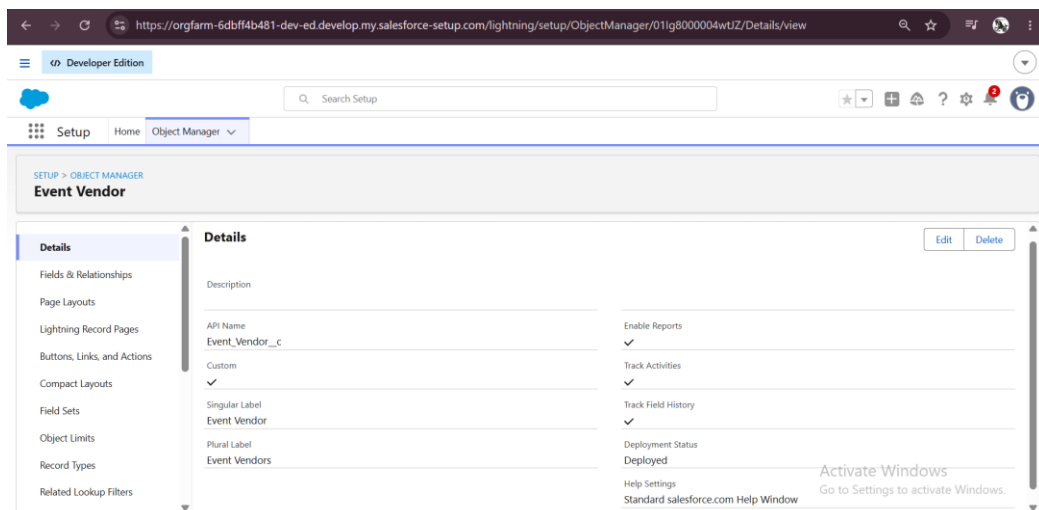


Figure 8: Event Vendor object details

8.4 Custom Tabs

A custom tab was created for every custom object (Clients, Events, Event Vendors, Feedbacks, Vendors and Venues) so that users can open each object from the navigation bar.

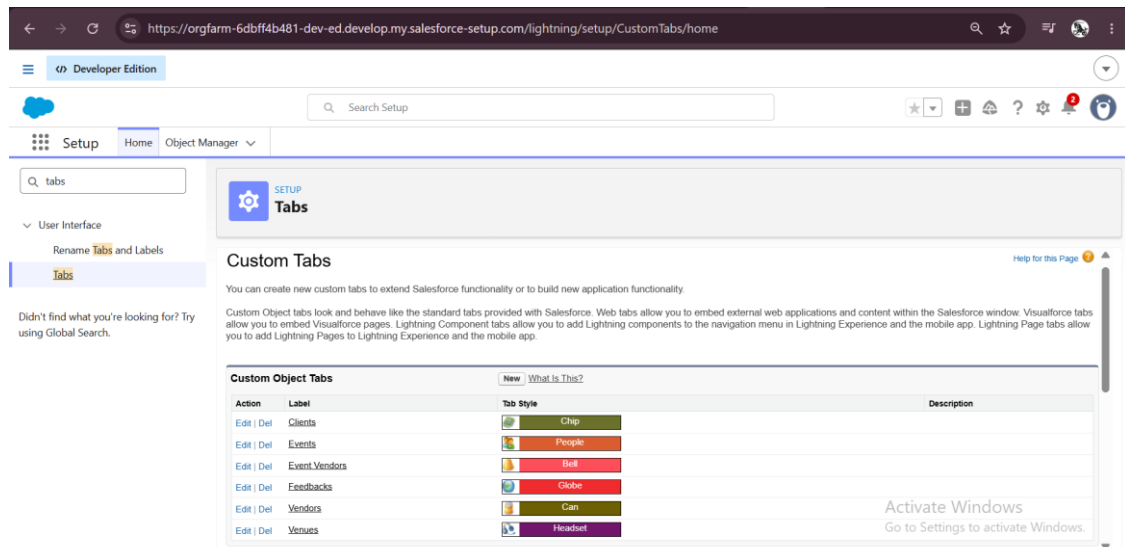


Figure 9: Custom Object Tabs in Setup

8.5 Fields

Object	Field	Data type / values
Event	Event Date	Date
Event	Event Type	Picklist: Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event	Event Status	Picklist: Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event	Event Budget	Formula (Currency), based on the event type
Event	Client, Venue	Lookup relationships
Client	Email, Phone, Address	Email, Phone, Text Area
Client	Country, City	Picklists
Vendor	Service Type	Picklist (Multi-Select): Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor	Email, Phone, Status	Email, Phone, Picklist: Available, Booked, Cancelled
Venue	Address, Location, Capacity	Text Area, URL, Number
Venue	Availability Status	Picklist: Available, Reserved, Not Available
Feedback	Rating, Comments	Picklist 1 to 5, Long Text Area
Feedback	Event, Client	Lookup relationships

Figure 10 shows the Salesforce Setup interface for the 'Event' object. The 'Fields & Relationships' section displays a table with 10 items, sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Client	Client_c	Lookup(Client)		✓
Created By	CreatedById	Lookup(User)		
Event Budget	Event_Budget_c	Formula (Currency)		
Event Date	Event_Date_c	Date		
Event Name	Name	Text(80)		✓
Event Status	Event_Status_c	Picklist		
Event Type	Event_Type_c	Picklist		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Venue	Venue_c	Lookup(Venue)		

Figure 10: Fields and relationships of the Event object

Figure 11 shows the Salesforce Setup interface for the 'Client' object. The 'Fields & Relationships' section displays a table with 9 items, sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Address	Address_c	Text Area(255)		
City	City_c	Picklist		
Client Name	Name	Text(80)		✓
Country	Country_c	Picklist		
Created By	CreatedById	Lookup(User)		
Email	Email_c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		
Phone	Phone_c	Phone		

Figure 11: Fields of the Client object

Figure 12 shows the Salesforce Setup interface for the 'Vendor' object. The 'Fields & Relationships' section displays a table with 8 items, sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Email	Email_c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Phone	Phone_c	Phone		
Service Type	Service_Type_c	Picklist (Multi-Select)		
Status	Status_c	Picklist		
Vendor Name	Name	Text(80)		

Figure 12: Fields of the Vendor object

Figure 13 shows the Salesforce Setup interface for the 'Venue' object. The 'Fields & Relationships' section displays a table of 8 fields. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. The fields are: Address (Text Area(255)), Availability Status (Picklist), Capacity (Number(18, 0)), Created By (Lookup(User)), Last Modified By (Lookup(User)), Location (URL(255)), Owner (Lookup(User, Group)), and Venue Name (Text(80)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Address	Address__c	Text Area(255)		
Availability Status	Availability_Status__c	Picklist		
Capacity	Capacity__c	Number(18, 0)		
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Location	Location__c	URL(255)		
Owner	OwnerId	Lookup(User, Group)		
Venue Name	Name	Text(80)		

Figure 13: Fields of the Venue object

Figure 14 shows the Salesforce Setup interface for the 'Feedback' object. The 'Fields & Relationships' section displays a table of 8 fields. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. The fields are: Client (Lookup(Client)), Comments (Long Text Area(32768)), Created By (Lookup(User)), Event (Lookup(Event)), Feedback Name (Auto Number), Last Modified By (Lookup(User)), Owner (Lookup(User, Group)), and Rating (Picklist).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Client	Client__c	Lookup(Client)		✓
Comments	Comments__c	Long Text Area(32768)		
Created By	CreatedById	Lookup(User)		
Event	Event__c	Lookup(Event)		✓
Feedback Name	Name	Auto Number		✓
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User, Group)		
Rating	Rating__c	Picklist		

Figure 14: Fields of the Feedback object

Event Budget formula field

The Event Budget is not typed in by the user. A formula field calculates it from the event type: Wedding 50,000; Corporate 30,000; Birthday 10,000; Anniversary 20,000; Festival 60,000; Concert 40,000; Other 15,000; and 0 when no type is chosen.

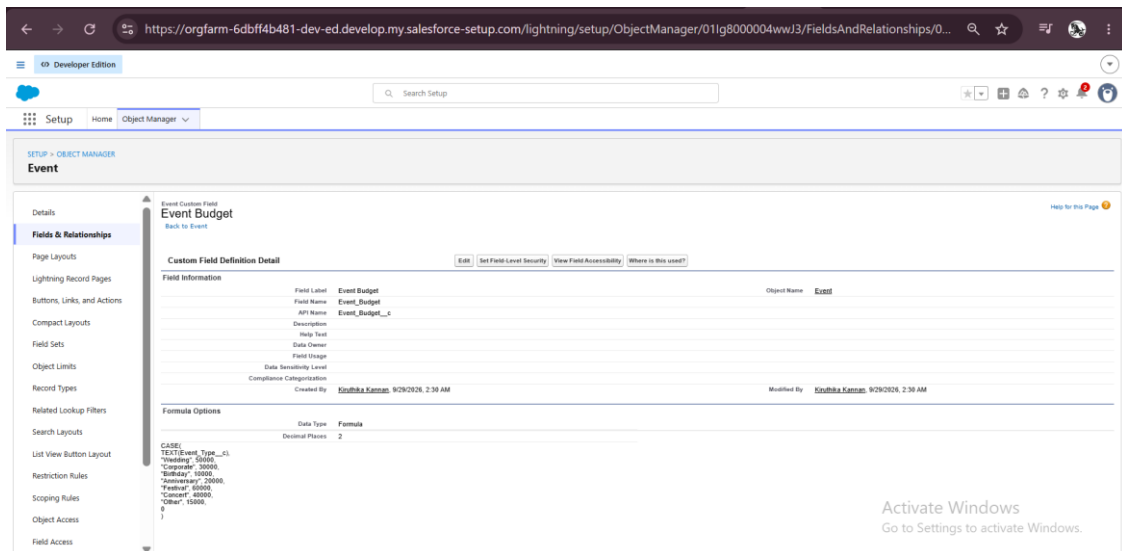


Figure 15: Event Budget formula field definition

8.6 Lookup Filter

On the Feedback object, the Event lookup was filtered so that only the events of the selected client can be chosen. The filter compares the Client on the Event record with the Client on the Feedback record, is marked Required and is enabled.

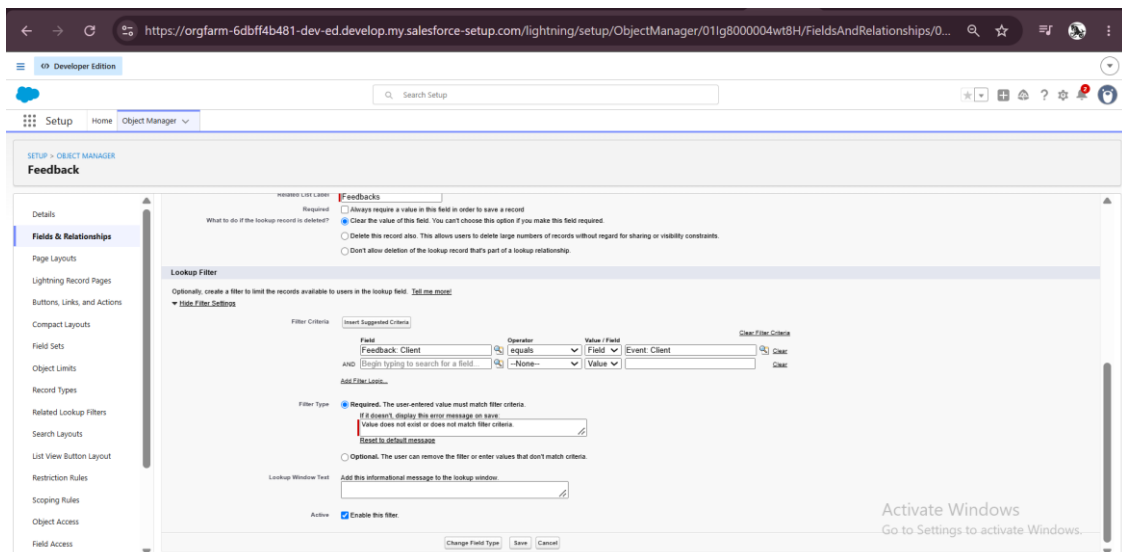


Figure 16: Lookup filter on the Feedback > Event field

8.7 Validation Rule

A validation rule named Email_Valid_Address was created on the Client object. It checks the email with a regular expression and blocks the save if the format is wrong. The error message "Please Enter Valid Email Address" is shown on the Email field.

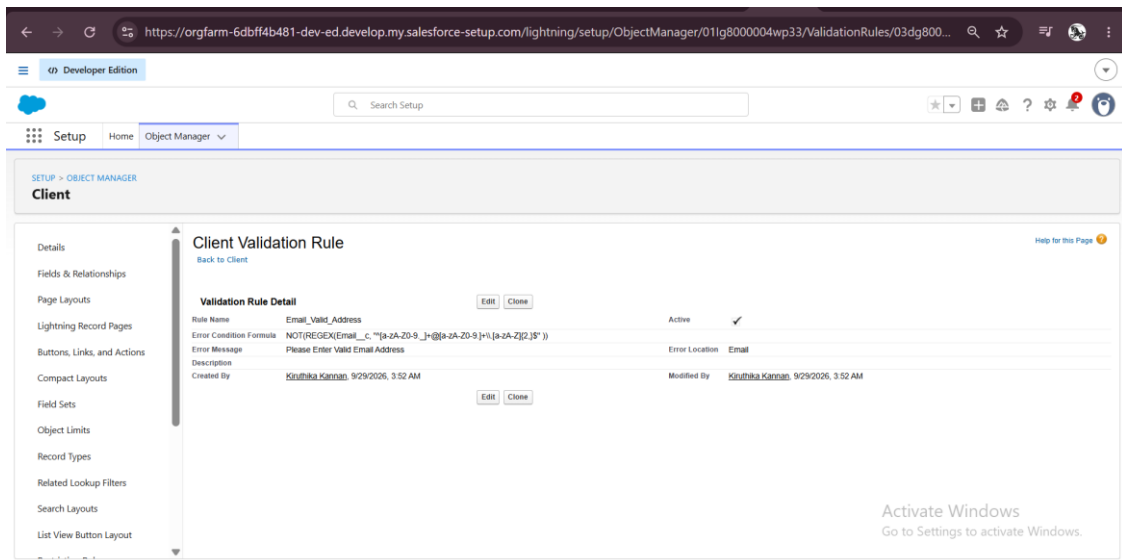


Figure 17: Validation rule Email_Valid_Address on the Client object

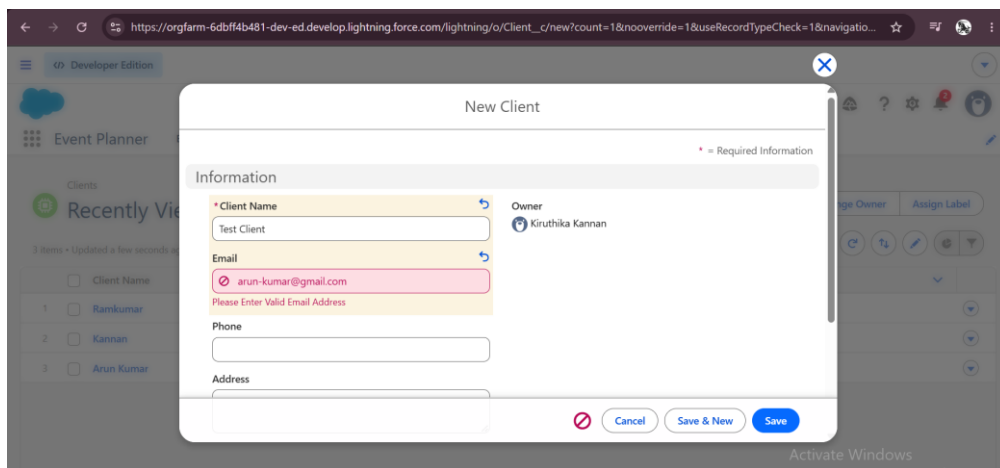


Figure 18: Error shown when an invalid email is entered

8.8 Approval Process for Event Cancellation

When a cancellation is requested, the request is sent to an approver instead of being accepted automatically. Email templates were created for the request, approval and rejection messages, and the approval process Event_Cancellation_Process was built on the Event object.

Setting	Value
Name	Event_Cancellation_Process
Object	Event
Entry criteria	Event Status equals Pending Cancellation
Record editability	Administrator only
Approval assignment email template	Event Cancellation Request Notification
Initial submitter	Event Owner
Initial submission actions	Record lock; email alert "Alert - Event Cancellation Request"

Setting	Value
Approval steps	Step 1 (manager of the submitter) and Step 2 "Manager Review" (assigned to a specific user)
Final approval actions	Record lock; field update Event Status = Canceled; email alert "Alert - Cancellation Approved"
Final rejection actions	Unlock the record; field update Event Status = Rejected

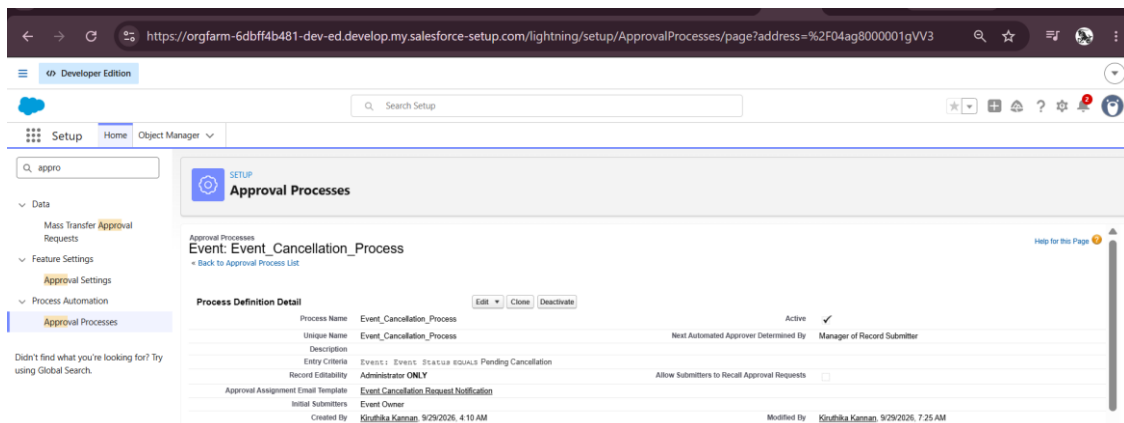


Figure 19: Approval process definition detail

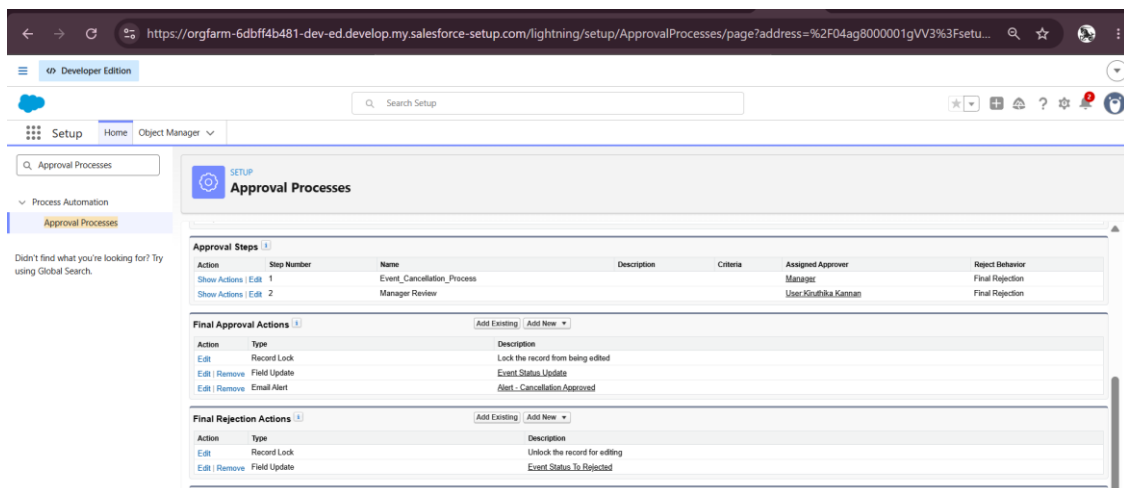


Figure 20: Approval steps, final approval actions and final rejection actions

8.9 Flow: Three-Day Client Reminder

A record-triggered Flow named Event_Reminder_Flow sends the client an email three days before an event.

1. Object: Event. Trigger: a record is created or updated. Condition: Event Status equals Confirmed.
2. A scheduled path named 3_Days_Before was added with the time source Event Date and an offset of 3 days before.
3. Text template resources named Subject and Body were created. The body greets the client by name and lists the event name, date, venue and type.

4. A Send Email action named Alert_Client_3Day_Reminder was added on the scheduled path. The recipient is the Client's email through the triggering Event record, and the event owner's email is in CC.
5. The Flow was saved and activated.

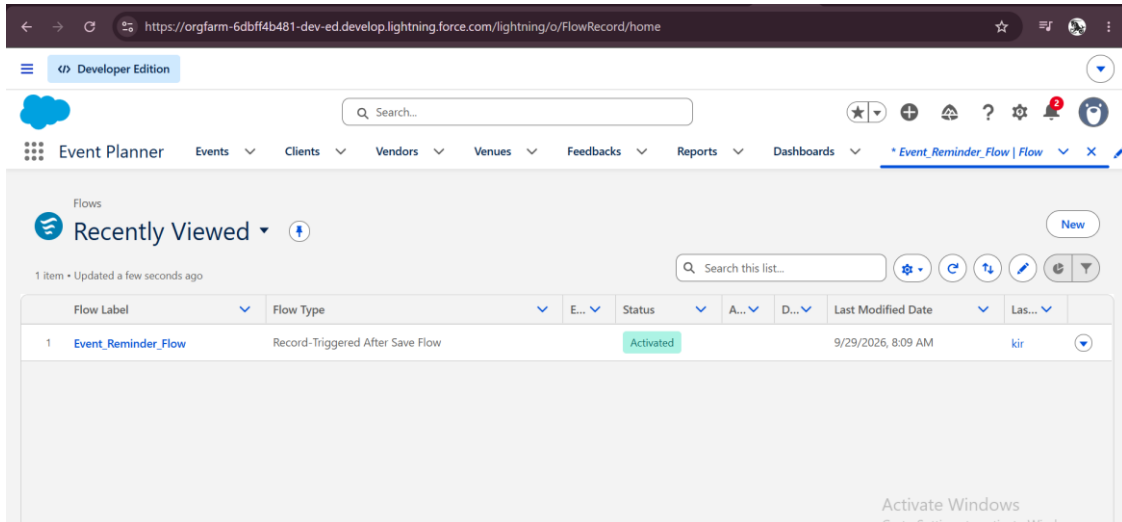


Figure 21: Event_Reminder_Flow shown as Activated

8.10 Apex Classes and Triggers

Apex was used where clicks were not enough. The complete code is available in Appendix A.

Venue availability (VenueStatusHelper and EventTrigger13)

When an event with a venue is saved as Confirmed, the venue's Availability Status becomes Reserved. When the event is Canceled, the venue becomes Available again. The helper class holds the logic and the trigger EventTrigger13 (after insert, after update) calls it.

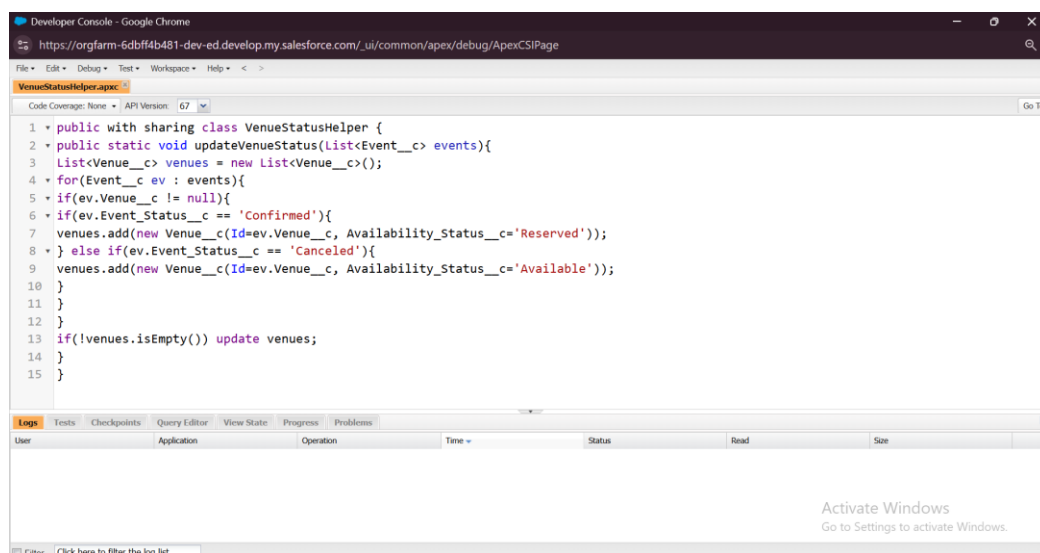


Figure 22: VenueStatusHelper class in Developer Console

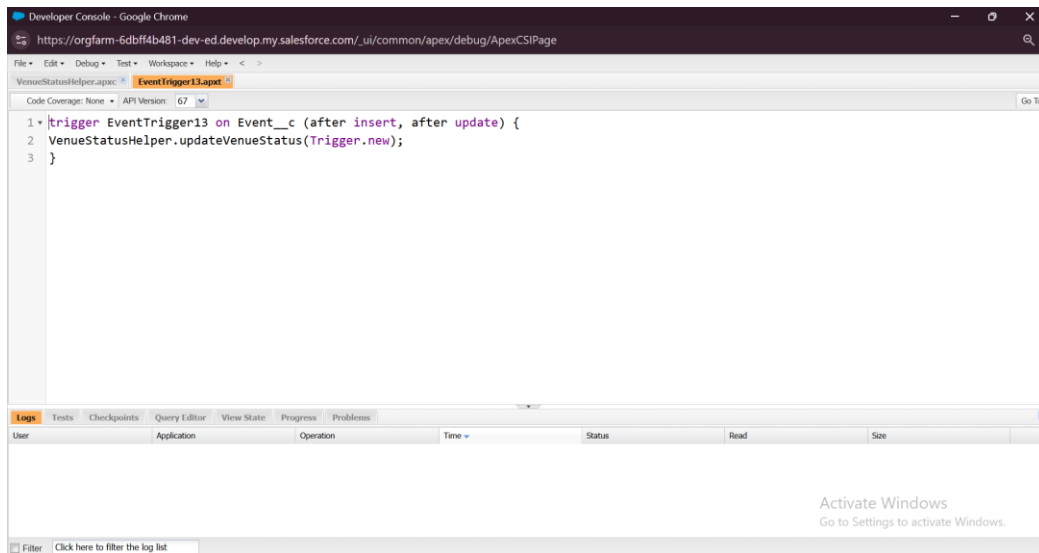


Figure 23: EventTrigger13 trigger in Developer Console

Double-booking prevention (PreventDoubleBooking)

This trigger runs before insert and before update. It collects the venues of the incoming events, reads other events at those venues and stops the save with the error "This Venue is already booked on this date." if another event exists on the same venue and date.

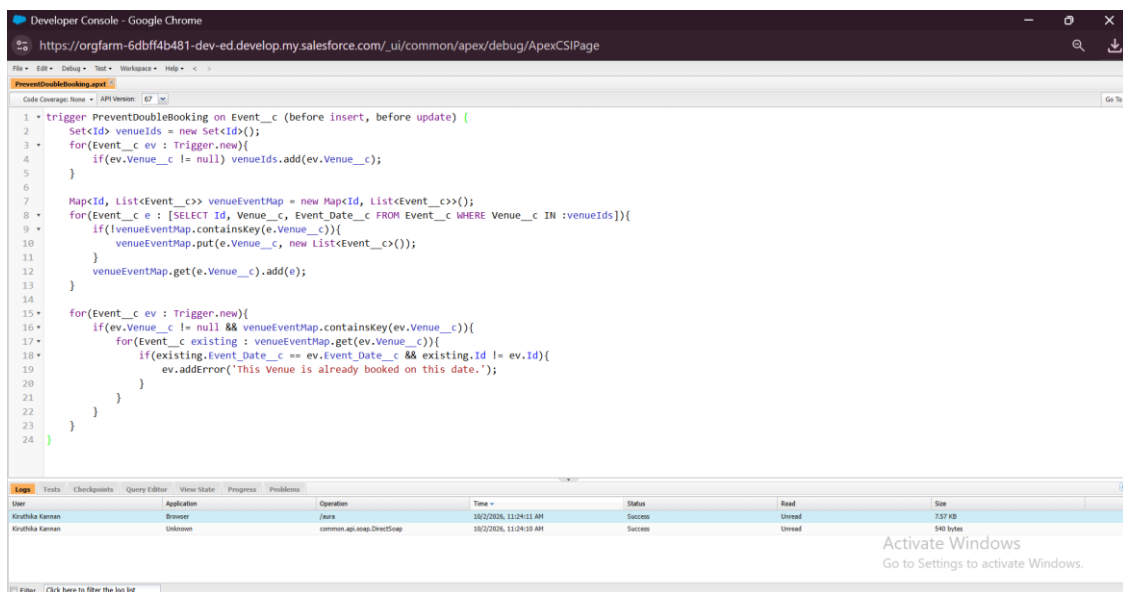


Figure 24: PreventDoubleBooking trigger

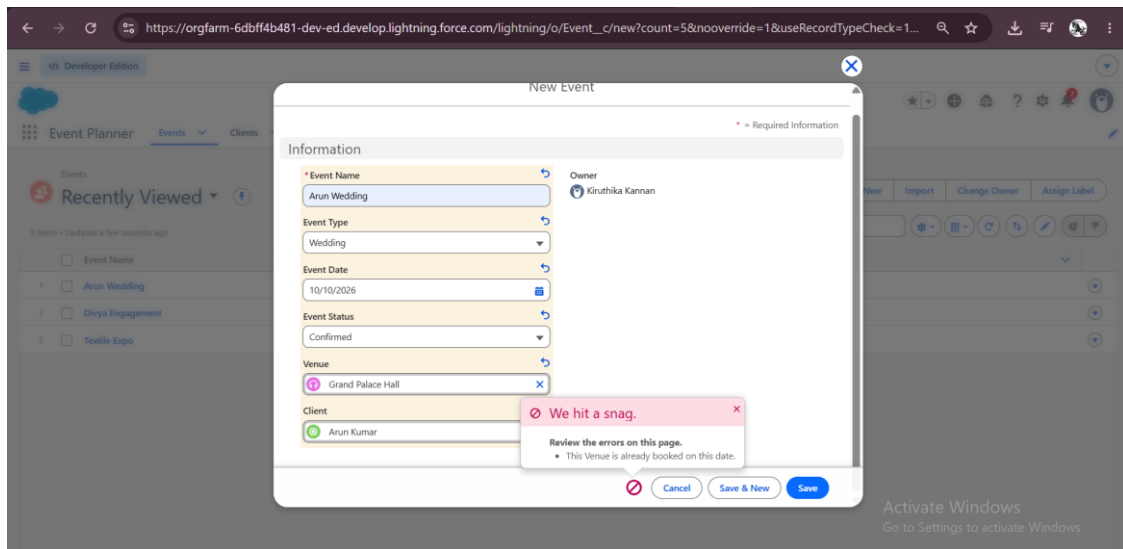
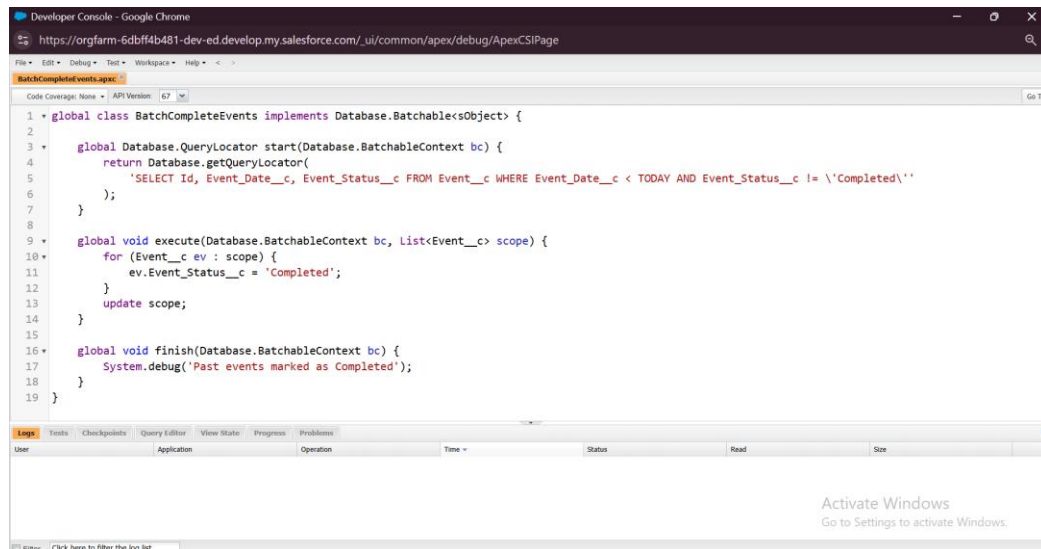


Figure 25: Error shown when the same venue is booked on the same date

Nightly completion of past events (Batch and Schedulable Apex)

BatchCompleteEvents selects events whose date is before today and whose status is not Completed, and sets their status to Completed. ScheduleCompleteEvents runs the batch in chunks of 200 records. The job Daily Event Completion was scheduled from Setup, Apex Classes, Schedule Apex, to run every day at 8:00 PM.



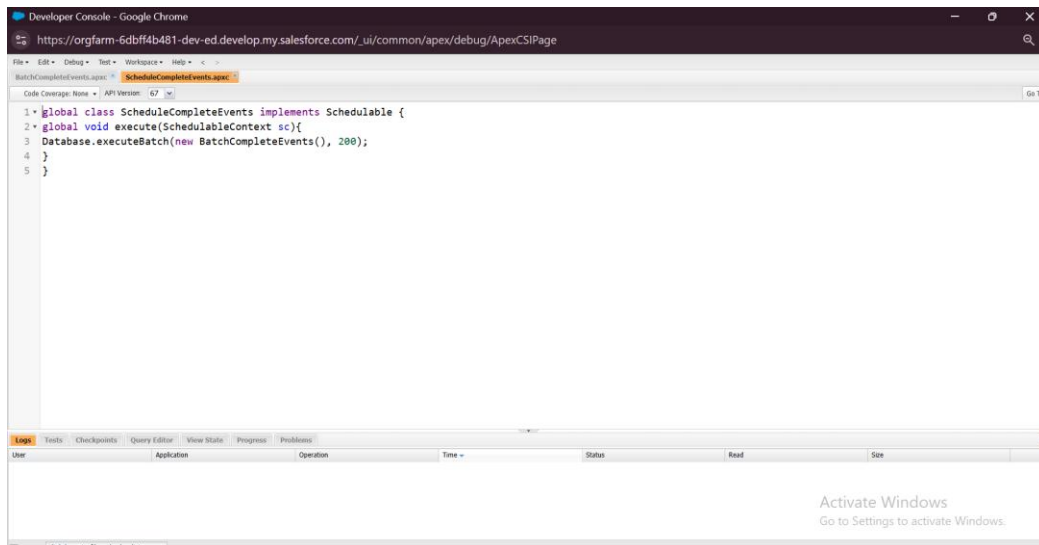


Figure 27: ScheduleCompleteEvents class in Developer Console

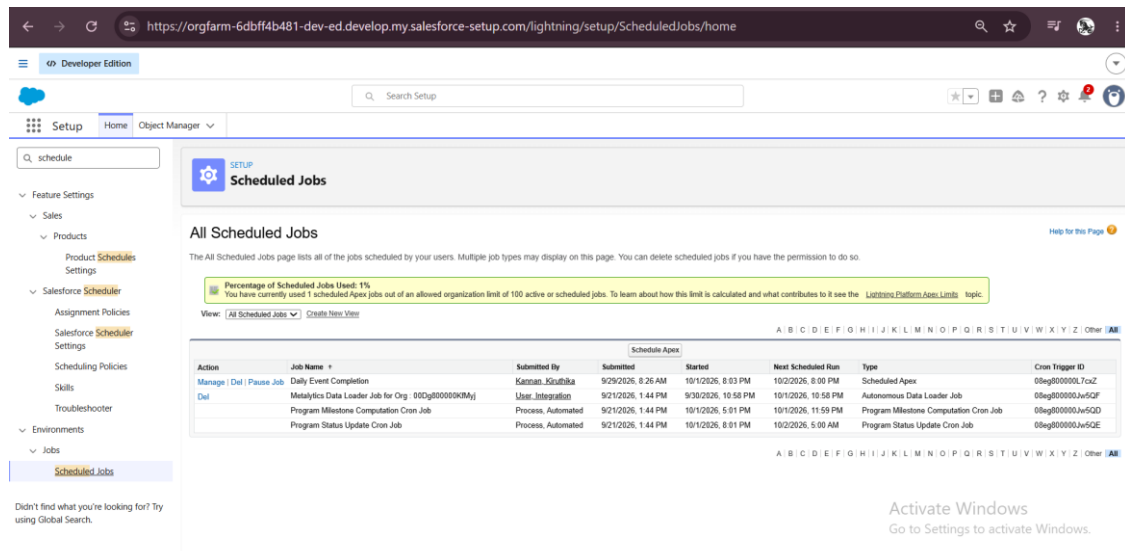


Figure 28: Scheduled job Daily Event Completion

9. Phase 3: UI/UX Development and Customization

This phase gave users a simple workspace and a clear view of the data.

9.1 Lightning App

A Lightning App named Event Planner was created from Setup, App Manager, New Lightning App. A logo image was uploaded and the navigation items Events, Clients, Vendors, Venues, Feedbacks, Reports and Dashboards were added. The app opens from the App Launcher.

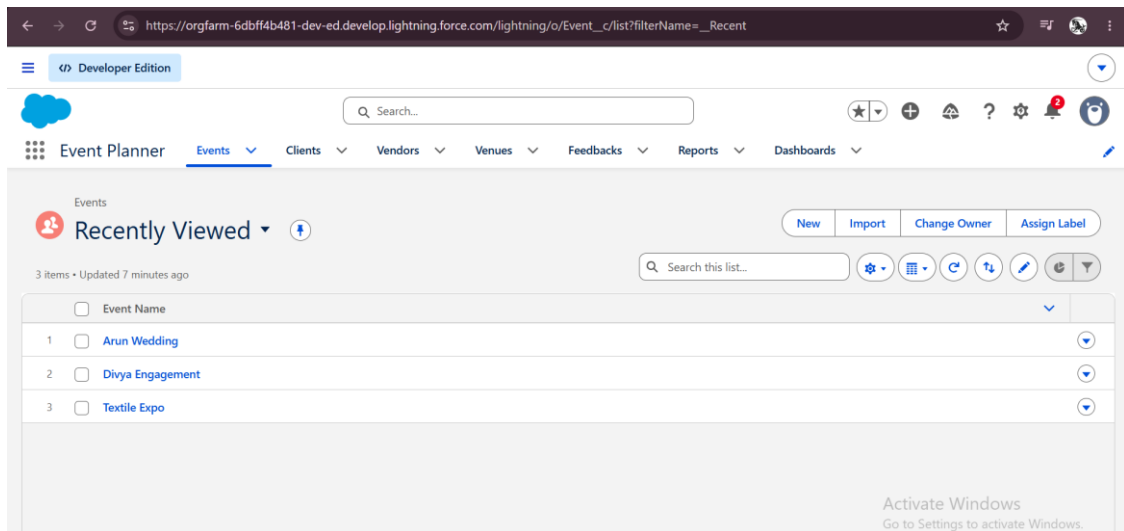


Figure 29: Event Planner app with its navigation tabs

9.2 Report

A report folder named EventForce Report was created. The report Upcoming Events by Month was built on the Events report type and saved in that folder.

Item	Setting
Columns	Event Name, Event Date, Event Type, Event Budget, Venue, Client
Filters	Event Date greater than or equal to TODAY; Event Status not equal to Canceled
Grouping	Event Date grouped by Calendar Month
Summary	Record count (row counts, subtotals and grand total enabled)
Chart	Bar chart of record count for each month

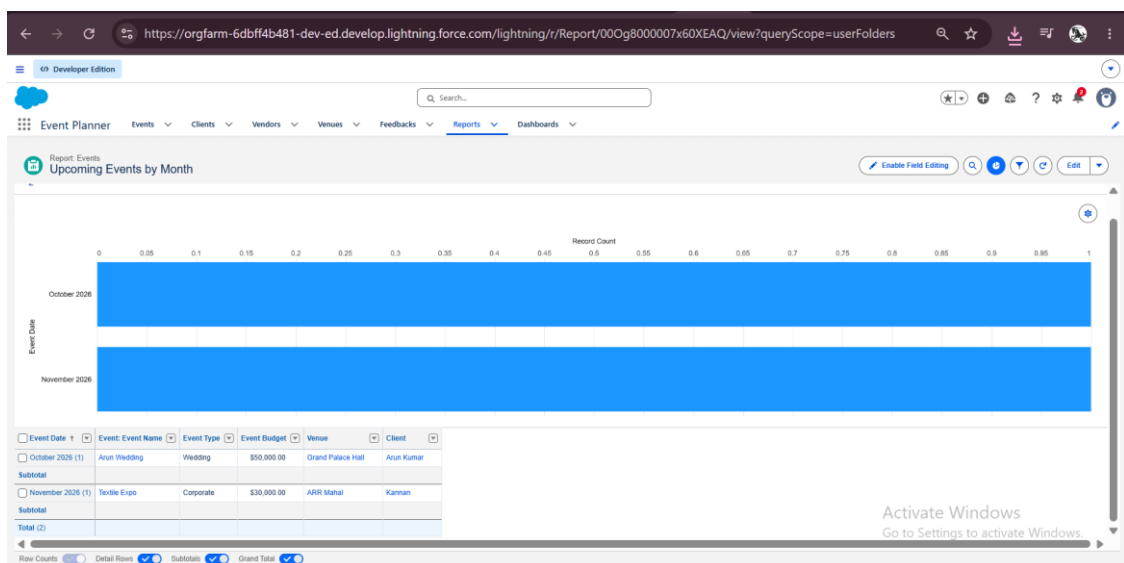


Figure 30: Upcoming Events by Month report with chart

9.3 Dashboard

The dashboard EventForce Operations Dashboard contains a donut chart widget based on the Upcoming Events by Month report, with a maximum of 6 values displayed, the subtitle "Upcoming Events Overview" and the light theme.

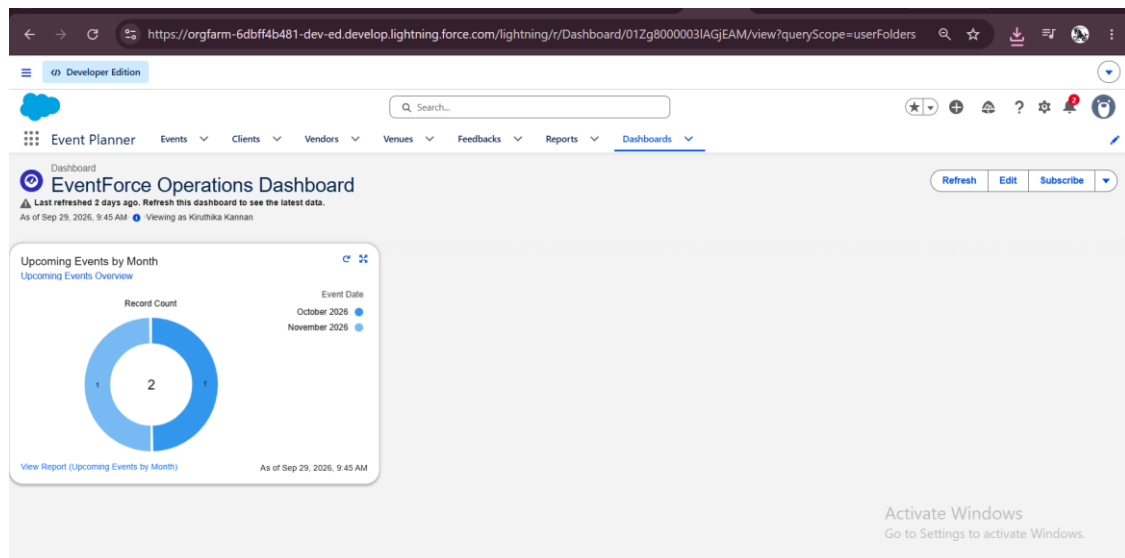


Figure 31: EventForce Operations Dashboard

10. Phase 4: Data Migration, Testing and Security

10.1 Sample Data

Sample records were created for the Event and Client objects so that every feature could be tested, for example the events Arun Wedding and Textile Expo that appear in the report and dashboard. For larger data sets the Data Import Wizard (Setup, Data Import Wizard, Custom Objects) can be used with CSV files, loading parent objects first so that lookups can be matched.

10.2 Roles

A role hierarchy was created under the organization name. The roles Event Admin, Vendor Manager, Client, Event Manager and Event Coordinator were added, so that users higher in the hierarchy can see the records of the users beneath them.

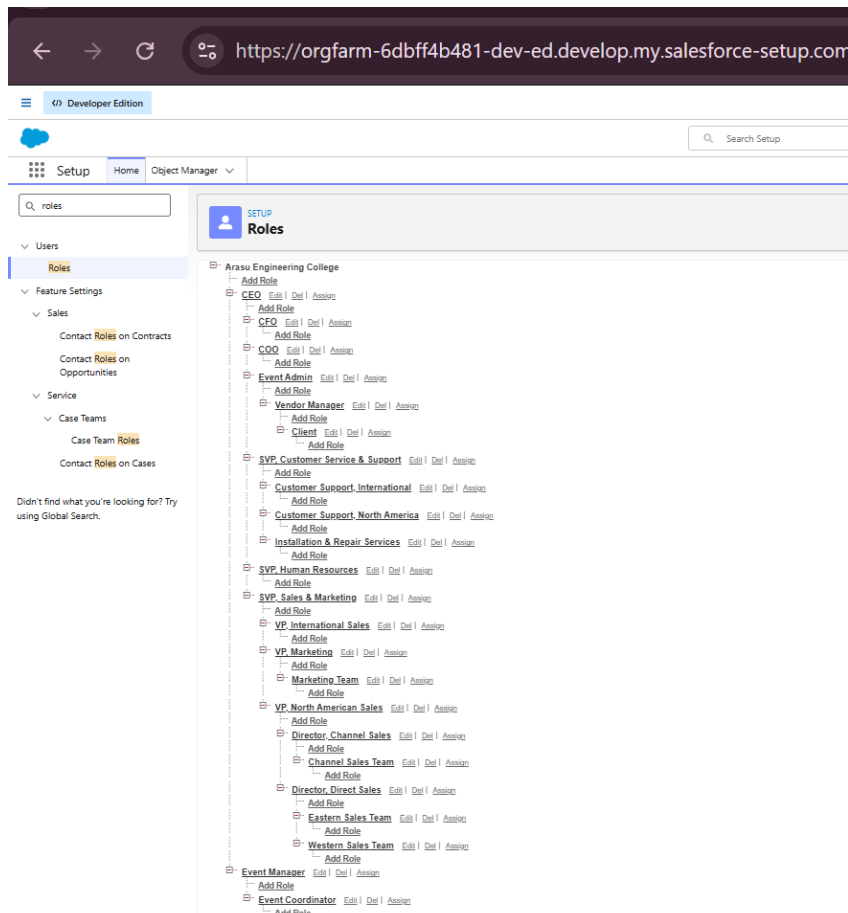


Figure 32: Role hierarchy

10.3 Profiles

Custom profiles Event Coordinator, Vendor Manager and Client Profile were created as clones of a standard profile, and each was given object access that matches the role.

Profile	Intended object access
Event Coordinator	Event: create, read, edit, delete. Client: read, edit. Vendor and Venue: read. Feedback: create, read.
Vendor Manager	Read-only access to Event, Venue, Client and Feedback.
Client Profile	Event: read only. Feedback: create, read. No access to Vendor and Venue.

10.4 Users

Users were created for the Event Coordinator, Vendor Manager and Client roles. Each user has the matching role and profile, as shown below. The administrator account keeps the System Administrator profile.

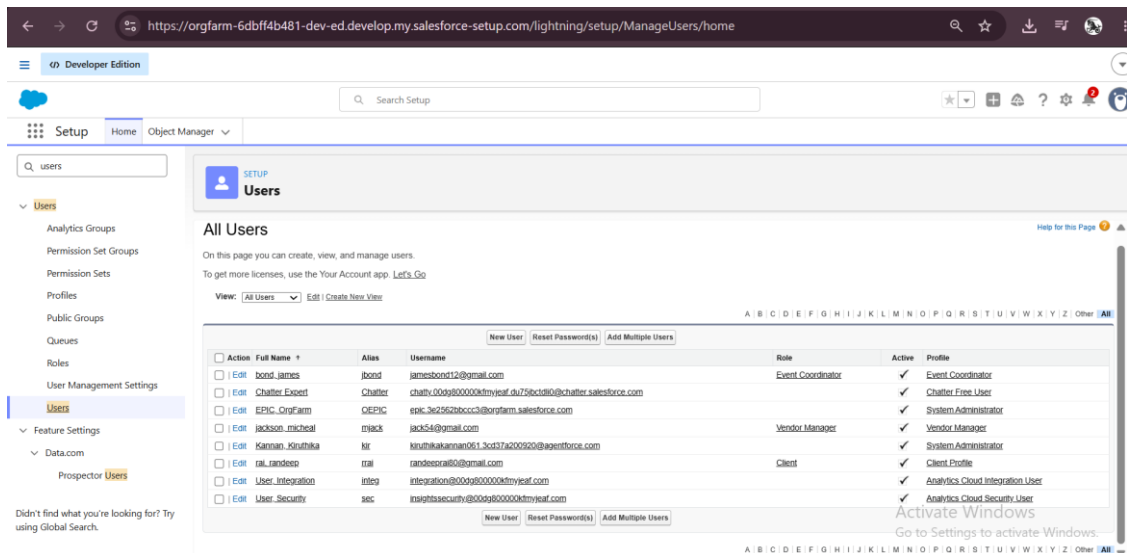


Figure 33: Users with their roles and profiles

10.5 Permission Set

A permission set named Feedback Manager was created to grant extra object permissions on Feedback (read, create, edit and delete) without changing a user's profile.

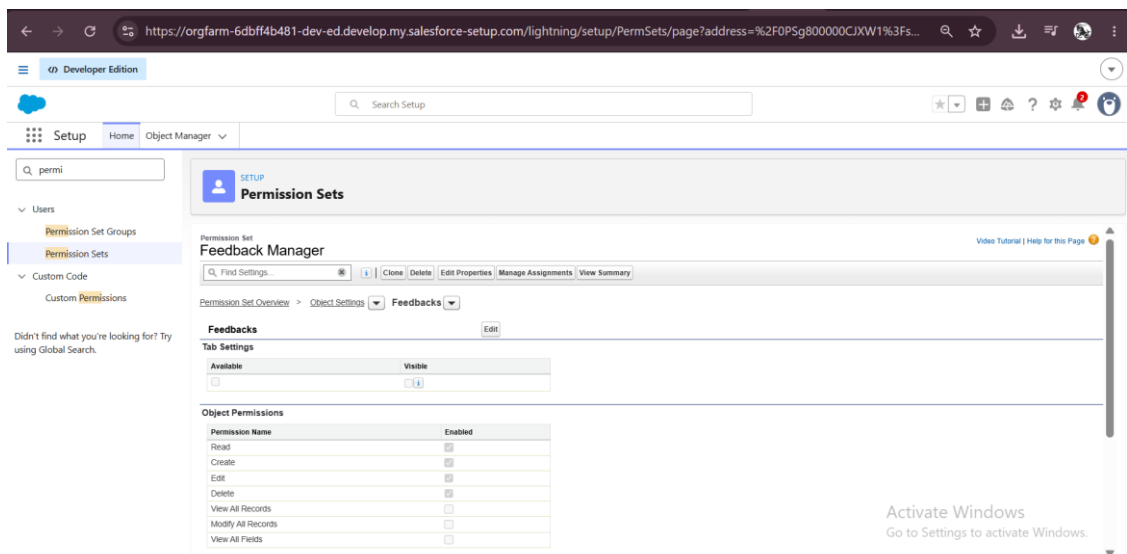


Figure 34: Feedback Manager permission set, object permissions for Feedbacks

10.6 Sharing Settings

The organization-wide default for the Event object was set to Private for internal and external access. Client, Feedback, Vendor and Venue are Public Read/Write internally. To let vendor managers see events, an owner-based sharing rule was created on Event: records owned by members of the Event Coordinator role are shared with the Vendor Manager role with Read Only access.

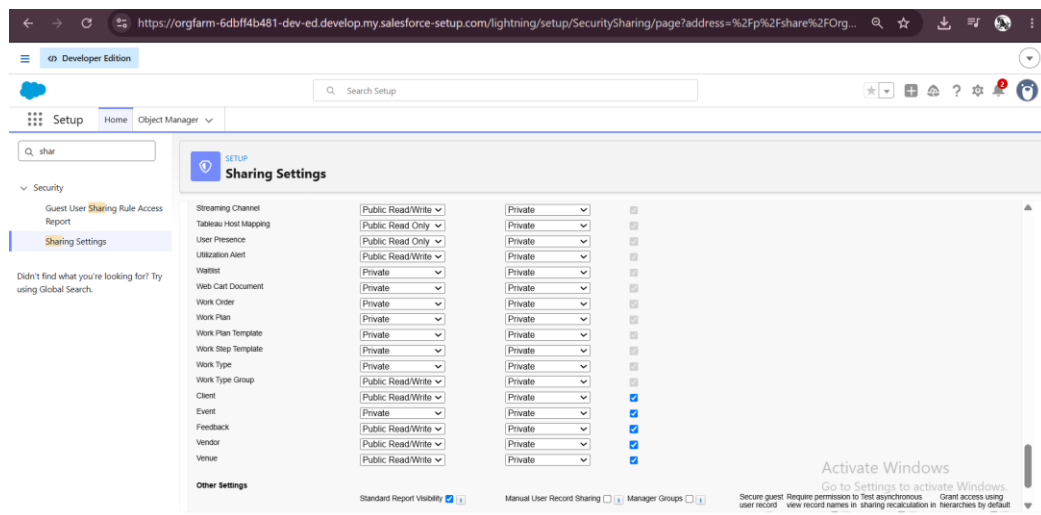


Figure 35: Organization-wide defaults

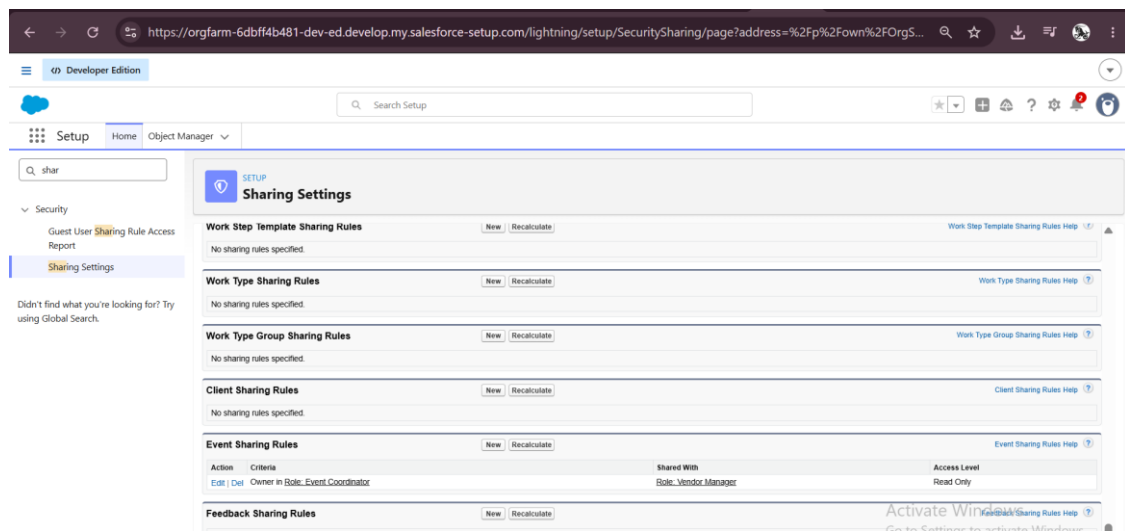


Figure 36: Event sharing rule: Event Coordinator to Vendor Manager (Read Only)

10.7 Testing

Each feature was tested after it was built. The results are summarized below.

No.	Test case	Expected result	Result
1	Save a client with an invalid email	Error: Please Enter Valid Email Address	Pass
2	Create events of type Wedding and Corporate	Event Budget shows 50,000 and 30,000	Pass
3	Book a second event at the same venue on the same date	Error: This Venue is already booked on this date.	Pass
4	Schedule the nightly Apex job	Job Daily Event Completion appears in Scheduled Jobs with a next run time	Pass
5	Open the report and dashboard	Events grouped by month; dashboard shows the donut chart	Pass

Functional testing evidence

Each result above was checked in the Salesforce user interface. The screenshots below show the outcome of the tests.

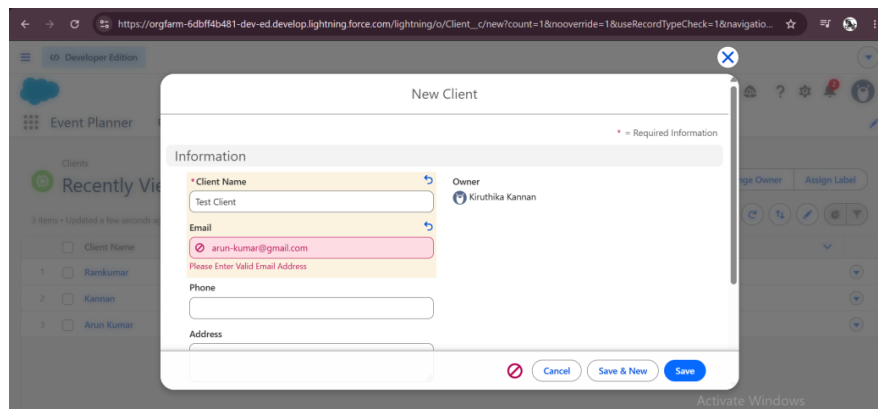


Figure 37: Test 1: invalid email blocked by the validation rule

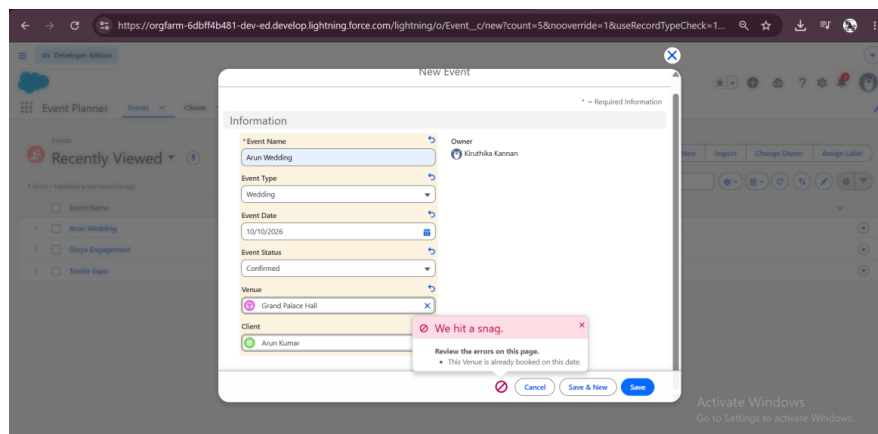


Figure 38: Test 3: double booking blocked by the Apex trigger

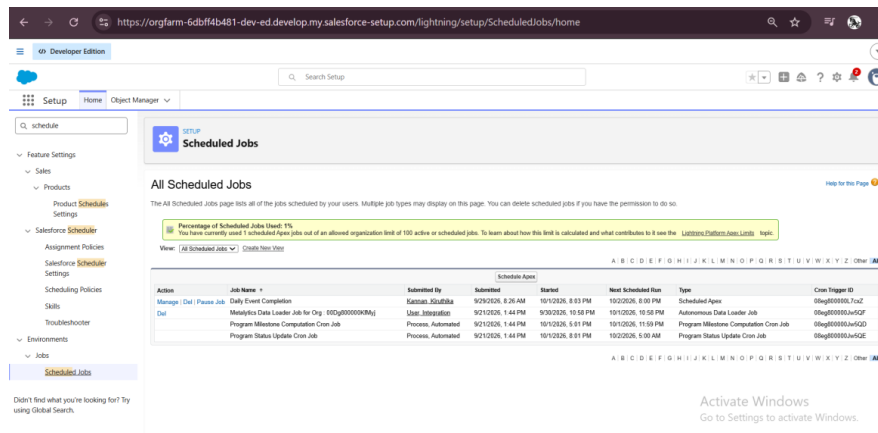


Figure 39: Test 4: nightly Apex job scheduled

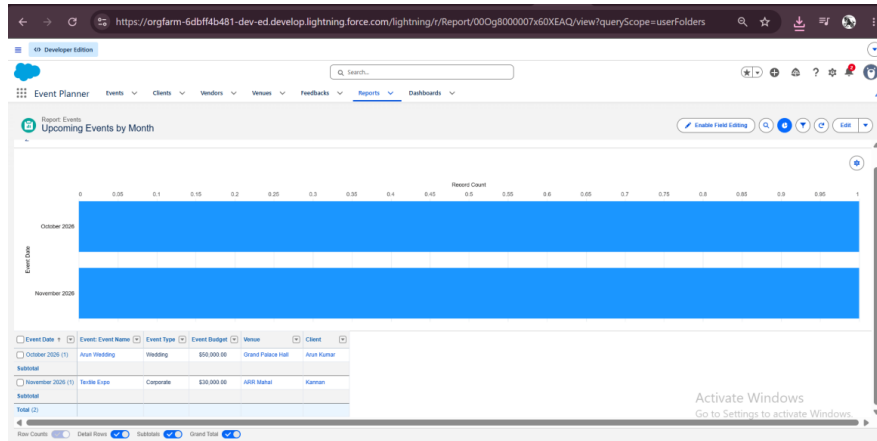


Figure 40: Test 5: report showing upcoming events by month

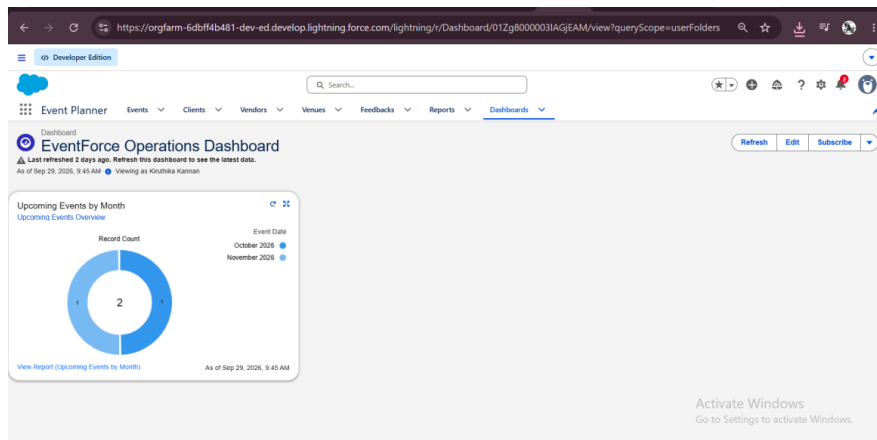


Figure 41: Test 5: dashboard showing the donut chart

Performance considerations

The Apex code is written for bulk processing. The triggers collect records in sets and maps, run one query and one update for the whole batch of records, and the nightly job processes records in chunks of 200, so the code stays within Salesforce governor limits.

11. Phase 5: Deployment, Documentation and Maintenance

11.1 Deployment

All configuration and code was built and tested in a Developer Edition org. A Developer Edition org is a standalone environment and is not connected to a production org, so a real production deployment was not carried out. In an enterprise setting the same components (objects, fields, flows, Apex, reports and dashboards) would be moved through a sandbox using change sets or Salesforce DX with version control such as GitHub. The code and documentation of this project are kept in a GitHub repository so that they are organized and ready for migration.

11.2 Maintenance, Monitoring and Troubleshooting

- Monitor Apex jobs, flows and approval processes regularly to catch failures early.

- Fix access problems by checking profiles, permission sets, roles and sharing rules.
- Debug triggers and flows with the Developer Console and debug logs.
- Update validation rules and sharing settings as business needs change.

11.3 Documentation

This document records the business purpose, the data model, the automation, the security model and the testing of the system, together with screenshots. It is intended to help future developers and administrators understand and extend the application.

12. Challenges Faced and Fixes

Problem	Cause	Fix
Apex error: Illegal string literal, line breaks are not allowed	The SOQL query inside BatchCompleteEvents was split over two lines.	The query string was written on a single line.
Send Email action in the Flow showed an error	The recipient address was empty.	The recipient was set to the triggering Event's Client email.
Approval step showed "You must enter a value"	The approver field was left blank.	A user was selected as approver with the lookup icon.
Report displayed "No Results"	Event records had dates in the past, so the TODAY filter excluded them.	Events with future dates were created and the report was refreshed.
Double-booking error did not appear	The PreventDoubleBooking trigger body was empty.	The full trigger code was pasted and saved, and the error then appeared.

13. Conclusion

The EventForce Management System shows how Salesforce can organize and automate the work of an event planning business. With a clear data model, a validation rule, an approval process, a reminder Flow, Apex logic, a Lightning App, a report and a dashboard, the application reduces manual effort and gives every user a clear, secure view of the data they need.

Building the project gave practical experience in requirement analysis, object and relationship design, declarative automation, Apex development, security configuration, testing and documentation. Although the project was completed in a Developer Org for learning, it reflects how a real Salesforce implementation is planned and delivered, and it can be extended in the future with production deployment, more reports and integrations.

Appendix A: Apex Source Code

VenueStatusHelper.cls

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

EventTrigger13.trigger

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Triiger.new);
}
```

PreventDoubleBooking.trigger

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }
    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c FROM Event__c WHERE
Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id !=
ev.Id){
                    ev.addError('This Venue is already booked on this date.');
```

BatchCompleteEvents.cls

```
global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c WHERE
Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
```



```

    );
}
global void execute(Database.BatchableContext bc, List<Event__c> scope) {
    for (Event__c ev : scope) {
        ev.Event_Status__c = 'Completed';
    }
    update scope;
}
global void finish(Database.BatchableContext bc) {
    System.debug('Past events marked as Completed');
}
}

```

ScheduleCompleteEvents.cls

```

global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}

```